Author: Trevor Robbins
Project: Calculating PI
Class: CS 431/531
Date: October 24, 2026

Analysis of OpenMP Parallelization for Calculating Digits of PI

In this report, I conducted a case study for a serial vs parallel implementation of the Riemann Sum Algorithm (RSA) calculating the first n-digits of pi. On top of this, the codebase includes an implementation of the random variable Monte Carlo Method (MCM) for calculating pi, counting digits that fall inside a circle. Thread counts split across the parallel implementation depended on available CPU-cores, that being 2, 4, 8, 16, 32, and 64.

For the Riemann Sum Algorithm, this involved integrating over a quarter circle for a sum of rectangular areas between the interval $[0, 1]$. This can be serially implemented, however partitioning the sums allows for a parallel implementation that splits rectangles across threads for the for loop itself.

Moving onto the Monte Carlo Method, this simple technique is more of an analog for an incredibly educated guess. Using a randomly generated number between the interval of $[0, 1]$, our threads "guess" for an area inside of a unit circle that is contained within a square. Guesses outside the square are discarded, however the remaining "right choices" are added to our running "hits" variable sum. For this algorithm, I decided to try a hybrid bit-shifting with hashing approach to see if a speedup could be induced.

Results

In the results section, we will analyze the performance speedup from serial to parallel. As well as this, we will be using the scaling plot and benchmark data shown below:

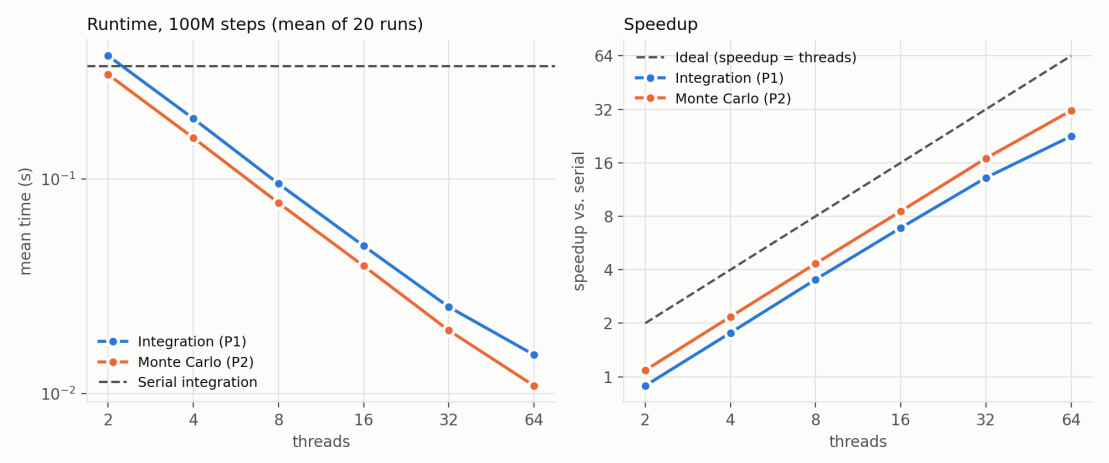


Figure 1: Runtime (100M steps, mean of 20 runs) and speedup comparison vs. ideal linear scaling.

Analyzing the results, the most surprising outcome was the massive speedup from the MCM to the RSA. Using the 64'th thread as the largest scaling example as thread counts increase, we

Table 1: Performance and speedup benchmarks across thread counts (100M steps, 20 runs).

Threads	**Serial (s)**	**RSA P1 (s)**	**P1 Speedup**	**MCM P2 (s)**	**P2 Speedup**	**P1 Error**
2	0.3352	0.3767	0.890×	0.3081	1.088×	1.021×10^{-11}
4	0.3393	0.1920	1.767×	0.1561	2.174×	1.021×10^{-11}
8	0.3363	0.0954	3.526×	0.0776	4.333×	1.021×10^{-11}
16	0.3365	0.0489	6.884×	0.0394	8.536×	1.021×10^{-11}
32	0.3345	0.0254	13.181×	0.0197	16.952×	1.021×10^{-11}
64	0.3423	0.0152	22.556×	0.0109	31.441×	1.021×10^{-11}

can notice that its an almost 10x speedup over the RSA parallelization. I believe the reason behind this performance boost is due to two reasons. One is likely CPU-partitioning overhead, as analyzing the serial vs parallel RSA, the total time per thread scales better for the serial version, likely due to the lack of cost for context switching. Another reason could be the use of bit shifting for random-number generation rather than library calls, which could have sped up the result more than hypothesized. Regardless, taking a look at Figure 1, results scale linearly for input-size to thread ratio.